Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No:2601089
Name: Vaishnavi Chandrashekhar Kagale

Practice Assignment 1

Ass1: Create a Singly Linear Linked List with following operations

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None
```

1. Create Linked List

```
    def create(self):
        n = int(input("How many nodes? "))
        for i in range(n):
            data = int(input("Enter value: "))
            self.insert_end(data)

    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node
```

2. Traverse and print the node values

```
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

3. Insert node at a specific position

```
    def insert_position(self):
        data = int(input("Enter value: "))
        pos = int(input("Enter position: "))
        new_node = Node(data)

        if pos < 1:
            print("Invalid position")
            return
```

```
                    if pos == 1:
            new_node.next = self.head
            self.head = new_node
            return

        temp = self.head
        for i in range(1, pos - 1):
            if temp is None:
                print("Position not found")
                return
            temp = temp.next

        if temp is None:
            print("Position not found")
            return

        new_node.next = temp.next
        temp.next = new_node
```

4. Find Middle node and print its value

```
def middle(self):
    if self.head is None:
        print("List is empty")
        return

    slow = self.head
    fast = self.head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

    print("Middle node:", slow.data)
```

5. Delete node

```
def delete(self):
    value = int(input("Enter value to delete: "))

    if self.head is None:
        print("List is empty")
        return

    if self.head.data == value:
        self.head = self.head.next
        print("Node deleted")
        return

    temp = self.head
    while temp.next is not None and temp.next.data != value:
        temp = temp.next

    if temp.next is None:
        print("Value not found")
    else:
        temp.next = temp.next.next
        print("Node deleted")
```

6. Reverse list

```
    def reverse(self):
        prev = None
        current = self.head

        while current is not None:
            next_node = current.next
            current.next = prev
            prev = current
            current = next_node

        self.head = prev
        print("List reversed")
```

7. Calculate the sum of every two consecutive node values.

```
    def pair_sum(self):
        temp = self.head

        if temp is None or temp.next is None:
            print("Need at least two nodes")
            return

        while temp.next is not None:
            print(temp.data, "+", temp.next.data, "=",
                  temp.data + temp.next.data)
            temp = temp.next
```

Main program

```
ll = LinkedList()

while True:
    print("\n1. Create Linked List")
    print("2. Display Linked List")
    print("3. Insert at Position")
    print("4. Find Middle Node")
    print("5. Delete Node")
    print("6. Reverse Linked List")
    print("7. Sum Consecutive Nodes")
    print("0. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        ll = LinkedList()
        ll.create()
    elif choice == 2:
        ll.display()
    elif choice == 3:
        ll.insert_position()
    elif choice == 4:
        ll.middle()
    elif choice == 5:
        ll.delete()
    elif choice == 6:
        ll.reverse()
```

```
                elif choice == 7:
    ll.pair_sum()
elif choice == 0:
    break
else:
    print("Invalid choice")
```